Interpolation, Curve Fitting and Calibration

Engineering Computation with Python — Chapter 12

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. Curve Fitting with Polynomials
6. EE Focus: The Bench Circuit's Time Constant
7. Summary

© Muním Zabidi

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. Curve Fitting with Polynomials
6. EE Focus: The Bench Circuit's Time Constant
7. Summary

Two Questions About Discrete Data

Measurements are taken at discrete points, but engineering questions rarely land exactly on one of them. This chapter covers two complementary answers.

Interpolation

Estimates values between measured points. Passes exactly through every known point.

Curve Fitting

Finds the single model that best describes a whole noisy dataset. Does not need to pass through any particular point.

The choice that matters

Clean, exact data → interpolate. Noisy, scattered measurements → fit. Curve fitting is the basis of **sensor calibration**.

Interpolation Versus Curve Fitting

Interpolation	Curve Fitting
Computes missing values between two known points.	Converts data points into a mathematical equation.
Original data points are preserved — the interpolant passes exactly through them.	The fitted curve does not need to pass through the original data points.

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. Curve Fitting with Polynomials
6. EE Focus: The Bench Circuit's Time Constant
7. Summary

The `interp1d` Syntax

One-dimensional interpolation in SciPy follows a consistent pattern: build an interpolating function once, then call it at any query points.

```
from scipy.interpolate import interp1d

# Define the interpolation function
f = interp1d(x, v, kind=method)

# Interpolated values at xq
vq = f(xq)
```

`numpy.interp()` does the same job for the linear case alone, with no object to build first.

`interp1d` Parameters

Parameter	Description
`x`	Sample points (array-like).
`v`	Corresponding values at the sample points.
`xq`	Points at which to evaluate the interpolation.
`method`	`'linear'`, `'nearest'`, `'cubic'`, etc.
`vq`	The interpolated values at `xq`.

The Linear Interpolation Formula

Every linear-interpolation call, under the hood, evaluates one formula. Given two known points (x_0, y_0) and (x_1, y_1), the value y at a query point x with $x_0 \le x \le x_1$ is

$$y = y_0 + (x - x_0)\,\frac{y_1 - y_0}{x_1 - x_0}.$$

What this is, geometrically

A straight line fitted between the two known points, evaluated at x. This is exactly what `numpy.interp()` and `interp1d(..., kind='linear')` compute.

Three Interpolation Methods

Method	Description	Min. Points
`nearest`	Steps to the value of the nearest data point.	2
`linear`	Straight lines connect the points.	2
`cubic`	Piecewise cubic spline.	4

Choosing a method is choosing an assumption

`nearest` assumes the signal holds its value until the next sample; `linear` assumes constant rate of change; `cubic` assumes smoothness. None of these is automatically correct — the right choice depends on the physics being measured.

Worked Example: Comparing Interpolation Methods

The same four points, interpolated three different ways:

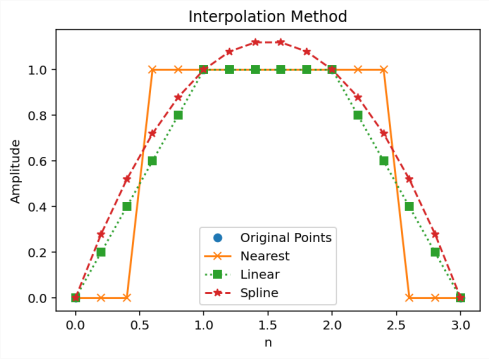
```
x = np.array([0, 1, 1, 0])
n1 = np.arange(0, 4)
n2 = np.arange(0, 3.2, 0.2)

xnearest = interp1d(n1, x, kind='nearest')
xlinear  = interp1d(n1, x, kind='linear')
xspline  = interp1d(n1, x, kind='cubic')

plt.plot(n1, x, 'o', label='Original Points')
plt.plot(n2, xnearest(n2), '-x', label='Nearest')
plt.plot(n2, xlinear(n2),  ':s', label='Linear')
plt.plot(n2, xspline(n2),  '--*', label='Spline')
```

© Muním Zabidi

Reading the Comparison Plot



Nearest produces a step-like response. **Linear** produces straight-line segments with visible corners. **Spline** produces a smooth curve — but notice it overshoots above 1 and below 0, values the original data never contained.

Table of Contents

1 Interpolation vs. Curve Fitting

2 1-D Interpolation in Python

3 **Worked Example: The API Haze Crisis**

4 Increasing Signal Detail

5 Curve Fitting with Polynomials

6 EE Focus: The Bench Circuit's Time Constant

7 Summary

The Air Pollutant Index

The API describes air quality from readings at monitoring stations, adopted by Malaysia's Department of Environment in 1996 (based on the US-EPA Pollutant Standards Index).

By the way: understanding the API scale

API Range	Health Status
0–50	Good
51–100	Moderate
101–200	Unhealthy
201–300	Very Unhealthy
301+	Hazardous

© Muním Zabidi

The Problem: A Haze Crisis

During a haze crisis, API readings are recorded at five stations along the route from Kulai to Johor Bahru:

Station	Kulai	Senai	Skudai	Perling	JB
Distance (km)	0	6	15	23	30
API value	100	180	199	280	310

The question

What is the API at a location with no sensor — say, kilometre 10? Assume the API varies linearly between neighbouring stations.

© Muním Zabidi

Building the Interpolation Function

A helper normalizes the input shape first:

```
arraykm = np.atleast_1d(km)
```

This accepts a single integer, a list, or a NumPy array uniformly — one code path, no type-checking branches.

```
def API_interpolation(km):
    distances  = np.array([0, 6, 15, 23, 30])
    api_values = np.array([100, 180, 199, 280, 310])
    full_distances = np.arange(0, 31, 1)
    interpolated_api = np.interp(full_distances,
                                 distances, api_values)
    km = np.atleast_1d(km)
    result = {}
    for k in km.astype(int):
        result[k] = round(interpolated_api[k], 2)
    return result
```

Calling It Three Ways

```
API_interpolation(10)                    # Single integer
API_interpolation([3, 8, 11, 26])        # List of km points
API_interpolation(np.linspace(0, 30, 5)) # Evenly spaced points
```

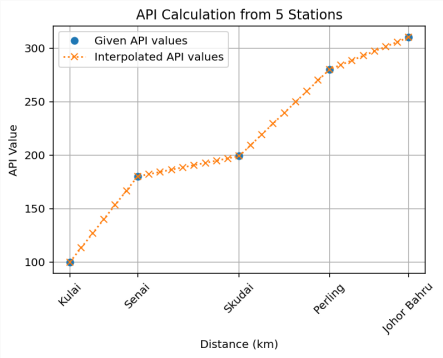
```
API value at km 10 is 188.44
```

```
API value at km 26 is 292.86
```

Notice

One function, three call shapes, zero extra code — the payoff of normalizing input with `np.atleast_1d` up front.

Visualizing the Interpolated Route



The five known stations (circles) anchor a straight-line estimate (dotted) for every kilometre in between.

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. **Increasing Signal Detail**
5. Curve Fitting with Polynomials
6. EE Focus: The Bench Circuit's Time Constant
7. Summary

Two broad uses recur throughout electrical engineering:

- Evaluating unknown values between known points — the API example just shown.
- Increasing the detail (resolution) of an undersampled signal.

Upsampling

Adding points *between* existing samples by interpolation is called upsampling. It cannot invent information the sensor never recorded — it can only estimate plausible values consistent with the samples that exist.

© Muním Zabidi

Worked Example: Sharpening an ECG Signal

A wireless ECG device under-samples to save energy. Spline interpolation adds points between the original samples:

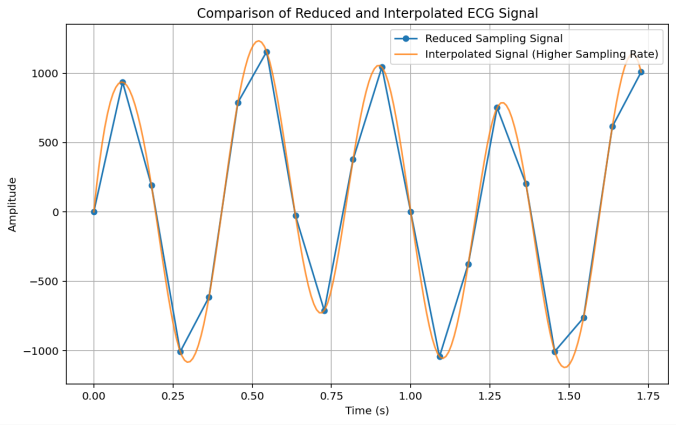
```
reduced_ecg_data = pd.read_csv('reduced_ecg_signal.csv')
time_reduced = reduced_ecg_data['Time(s)'].values
amplitude_reduced = reduced_ecg_data['Amplitude'].values

spline_interpolation = interp1d(time_reduced,
                                amplitude_reduced, kind='cubic')

new_time_high = np.linspace(time_reduced.min(),
                            time_reduced.max(),
                            len(time_reduced) * 10)
new_amplitude_high = spline_interpolation(new_time_high)
```

© Muním Zabidi

Before and After: ECG Resolution



The original 100-point recording (markers) versus the 10×-upsampled spline interpolation (smooth line) — the same heartbeat, described with far more detail between samples.

© Munim Zabidi

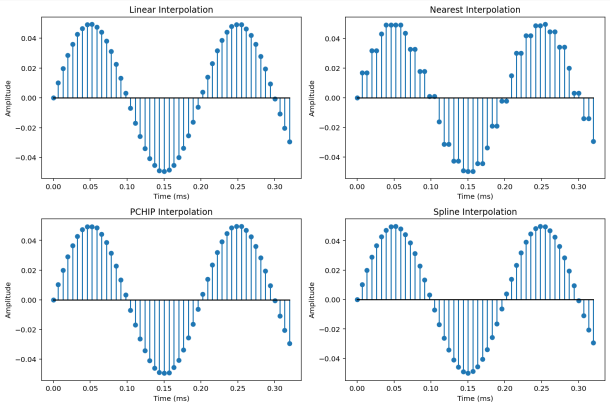
Worked Example: DAC Upsampling

A digital-to-analog converter (DAC) — the final step in playing digital audio — benefits from interpolation *before* conversion, simplifying the lowpass filter that follows:

$$\text{Audio Signal} \rightarrow \text{Interpolation (upsampling)} \rightarrow \text{Lowpass Filter} \rightarrow \text{DAC}$$

- **Linear**: straight lines between points.
- **Nearest**: step-like, assigns the nearest sample.
- **PCHIP**: smooth, no overshoot, preserves monotonicity.
- **Spline**: smooth, continuous derivatives, can overshoot.

Comparing Four Methods on a DAC Signal



Linear and spline look visually similar; nearest shows the characteristic steps; PCHIP tracks the data closely without spline's overshoot risk.

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. **Curve Fitting with Polynomials**
6. EE Focus: The Bench Circuit's Time Constant
7. Summary

A Straight-Line Trendline

Given scattered, noisy measurements, what single line $y = mx + b$ best describes the trend? `np.polyfit()` answers this directly:

```
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([1.1, 2.9, 5.2, 6.8, 9.3, 11.0])

m, b = np.polyfit(x, y, 1)
print(f"Best-fit line: y = {m:.2f}x + {b:.2f}")

y_fit = np.polyval([m, b], x)
```

```
Best-fit line:  y = 1.99x + 1.03
```

Unlike interpolation, this line does not pass through any of the original noisy points exactly — it summarizes the trend.

© Muním Zabidi

Fitting Higher-Degree Polynomials

The same `polyfit`/`polyval` pair scales to any degree:

```
p = np.polyfit(x, y, 3)  # Fit a 3rd-degree polynomial
y_fit = np.polyval(p, x) # Evaluate it
```

Why model with a polynomial at all?

Automating a process, analyzing the relationship between variables, and identifying trends — e.g. modelling yearly climate data or internet-usage growth.

Worked Example: Sensor Calibration

A temperature sensor reads in volts over 20–50°C. Find the function converting volts to temperature:

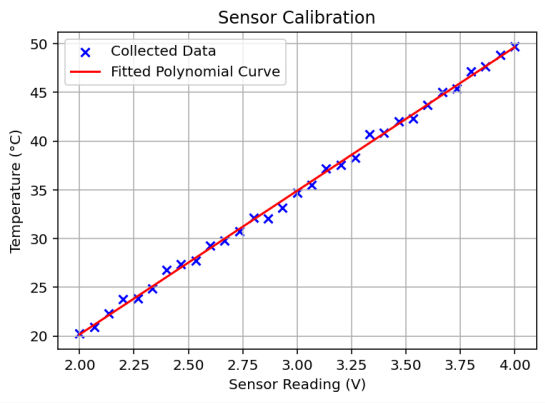
```
df = pd.read_excel('sensor_calibration.xlsx')
V, T = df['V'].values, df['T'].values

p = np.polyfit(V, T, 1)   # linear fit
s_t = np.polyval(p, V)
print(f'T(V) = {p[0]:.2f}V + {p[1]:.2f}')
```

```
T(V) = 14.74V + -9.33
```

This is sensor calibration: one `polyfit()` call turns a raw voltage reading into a trustworthy temperature.

The Calibration Curve



Collected data (crosses) with the fitted linear calibration curve $T(V) = 14.74V - 9.33$ (line).

Worked Example: Internet Usage Trend — the Data

Six months of internet usage (hours) for five first-time users. Task: find the trend and predict months 7–9.

Person	M1	M2	M3	M4	M5	M6
1	7	11	23	59	120	180
2	3	8	20	48	88	140
3	4	11	21	58	128	195

Fit 1st-, 2nd-, and 3rd-degree polynomials and compare their predictions.

© Muním Zabidi

Fitting Three Degrees at Once

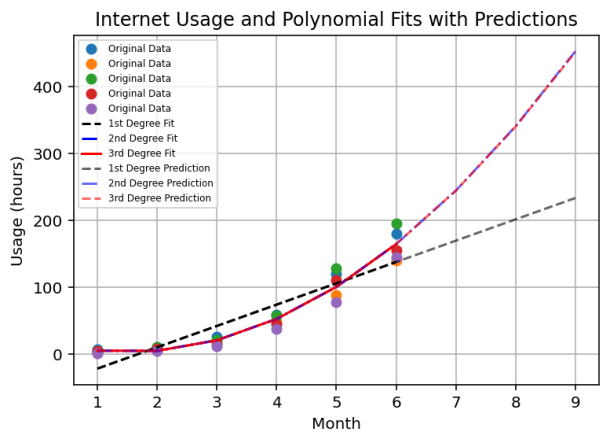
```
months = np.arange(1, 7)
month_matrix = np.tile(months, (data.shape[0], 1))

p1 = np.polyfit(month_matrix.flatten(), data.flatten(), 1)
p2 = np.polyfit(month_matrix.flatten(), data.flatten(), 2)
p3 = np.polyfit(month_matrix.flatten(), data.flatten(), 3)

months_predict = np.array([7, 8, 9])
usage_1 = np.polyval(p1, months_predict)
usage_2 = np.polyval(p2, months_predict)
usage_3 = np.polyval(p3, months_predict)
```

```
1st order:  [169.7 201.6 233.4]  2nd order:  [244.5 340.5 452.4]  3rd order:  [244.3 340.0 451.4]
```

Choosing a Degree: The Plot Tells the Story



The 1st-degree (linear) fit substantially underestimates the accelerating growth. The 2nd- and 3rd-degree fits track it closely and agree with each other.

Worked Example: Water Usage and R^2

A farmer wants an automatic irrigation formula from one harvest cycle's water-usage data. Which polynomial degree is the right choice?

R^2: how much variation the fit explains

$$R^2 = 1 - \frac{\sum_i (d_i - f_i)^2}{\sum_i d_i^2}$$

d_i = measured data, f_i = fitted data. Closer to 1 is better — but not the whole story.

Computing R^2 for Five Degrees

```
N = 5
water_fit = [np.polyval(np.polyfit(day, water, n), day)
             for n in range(1, N + 1)]

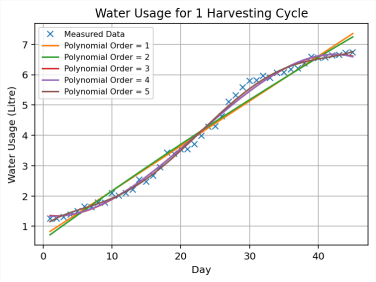
R2 = [1 - np.sum((fit - water)**2) / np.sum((water - np.mean(water))**2)
      for fit in water_fit]
```

```
Order 1 = 0.9743  Order 2 = 0.9750  Order 3 = 0.9947  Order 4 = 0.9947  Order 5 = 0.9961
```

Diminishing returns

R^2 jumps from 0.975 to 0.995 going from order 2 to 3, then barely moves beyond that. Order 3 is the sensible choice: good accuracy, no unnecessary complexity.

Why Not Always Pick the Highest R^2?



Overfitting

A polynomial with enough degrees can track *every wiggle* in noisy data, including the noise itself — it then describes only this particular sample, and predicts poorly on new data.

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. Curve Fitting with Polynomials
6. **EE Focus: The Bench Circuit's Time Constant**
7. Summary

The nominal time constant of the bench RC circuit is $\tau = RC = 0.47$ s — but that assumes the resistor and capacitor are exactly their marked values. Real components carry tolerances.

The honest answer comes from the data

What is the measured τ, from the actual voltage recording? This is curve fitting applied to the chapter's own running example.

The Trick: Logarithms Make It Linear

$v(t) = V_0 e^{-t/\tau}$ is not a straight line, and `np.polyfit()` fits polynomials. Taking logarithms of both sides:

$$\ln v = \ln V_0 - \frac{t}{\tau}$$

This *is* a straight line in t

Slope $= -1/\tau$, intercept $= \ln V_0$. Fitting a degree-1 polynomial to $\ln v$ recovers both parameters at once.

Recovering τ from the Data

```
t = np.arange(0, 2.001, 0.02)
v = 12.0 * np.exp(-t / 0.47)   # stands in for measured voltage

slope, intercept = np.polyfit(t, np.log(v), 1)

tau_measured = -1 / slope
V0_measured  = np.exp(intercept)
```

```
Fitted slope:  -2.12766 per second
Measured tau:  0.4700 s
Measured V0:  12.0000 V
```

Three lines of code measure the circuit's actual *RC* product, rather than assuming the marked component values.

© Muním Zabidi

Two Cautions Worth Carrying Forward

1. The logarithm is undefined for $v \leq 0$

Any sample that noise has driven to zero or below must be excluded *before* fitting.

2. Logarithms distort the error weighting

They compress large voltages and stretch small ones, so the tail of the decay influences the fit more than it would in a direct fit. Fine for a first estimate; for precision work, a nonlinear fit to the original exponential is the better tool.

Table of Contents

1. Interpolation vs. Curve Fitting
2. 1-D Interpolation in Python
3. Worked Example: The API Haze Crisis
4. Increasing Signal Detail
5. Curve Fitting with Polynomials
6. EE Focus: The Bench Circuit's Time Constant
7. **Summary**

Chapter Summary

- **Interpolation** passes exactly through measured points; **curve fitting** finds the best model for a noisy whole dataset. Noisy data calls for fitting, not interpolation.
- `np.interp()` does linear interpolation directly; `interp1d()` adds `nearest`, `cubic`, and spline methods via `kind`.
- The method must suit the physics: nearest $\rightarrow$ steps, linear $\rightarrow$ corners, cubic/PCHIP $\rightarrow$ smooth curves (but spline can invent overshoot the data never contained).
- `np.polyfit(x, y, n)` fits a degree-*n* polynomial; `np.polyval()` evaluates it. Degree 1 is a straight-line trendline — the basis of sensor calibration.
- R^2 measures explained variation, but always rises with degree. A polynomial that tracks every wiggle is overfitted and predicts poorly.
- Not every model is a polynomial: an exponential decay becomes linear under logarithms, recovering the bench circuit's τ with the same `polyfit()` tool.

© Muním Zabidi

This chapter answered *what value lies between* (or *near*) the data you have. The next chapters build on exactly this foundation.

- Chapter 13: numerical calculus — derivatives and integrals computed directly from arrays, and polynomial models of system behaviour.
- The bench circuit's measured τ, found here by curve fitting, reappears as a check against the calculus-based methods in the next chapter.

Next: Numerical Calculus and System Models

© Muním Zabidi